

UNIVERSITÀ DI BOLOGNA

Corso di Laurea in Ingegneria e scienze informatiche

Predictive Auto-Scaling in Kubernetes con Prophet e KEDA

Corso di Virtualizzazione e Integrazione di Sistemi

Studente

Francesco Meloni

Matricola: 00011144018

Anno Accademico

2025/2026

Indice

1	Introduzione	3
1.1	Obiettivi	3
2	Architettura del Sistema	4
2.1	Visione d'insieme	4
2.2	Struttura del progetto	4
2.3	Componenti applicativi	5
2.3.1	Random Service	5
2.3.2	Hash Service	5
2.3.3	Forecast Service	5
2.3.4	Load Generator	5
2.4	Infrastruttura Kubernetes	6
2.4.1	Networking e Ingress	6
2.4.2	Prometheus	6
2.4.3	KEDA	6
3	Implementazione	7
3.1	Microservizi Flask	7
3.2	Forecast Service	8
3.2.1	Raccolta dati da Prometheus	8
3.2.2	Modello Prophet con stagionalità personalizzata	8
3.2.3	Generazione della previsione	9
3.2.4	Loop principale	10
3.3	Configurazione KEDA	10
3.3.1	ScaledObject	10
3.4	Generatore di Carico	11
4	Risultati Sperimentali	12
4.1	Profilo di traffico generato	12
4.2	Metriche predette dal Forecast Service	13
4.3	Comportamento dello scaling	13

4.3.1	Numero di repliche nel tempo	13
4.3.2	Screenshot kubectrl	13
4.4	Interfacce utente dei servizi	15
4.5	Analisi del modello Prophet	15
4.5.1	Stagionalità rilevata	15
4.5.2	Scelta di <code>yhat_upper</code>	16
5	Conclusioni	17
5.1	Possibili evoluzioni	17

Capitolo 1

Introduzione

Nelle architetture cloud-native moderne, la capacità di adattare dinamicamente le risorse computazionali al carico di lavoro è un requisito fondamentale. L'approccio tradizionale, quindi il *reactive auto-scaling*, che scala i servizi *in risposta* a metriche già misurate, ha un problema: le nuove istanze vengono avviate solo dopo che il picco di traffico si è già manifestato, quindi si hanno dei momenti in cui si possono avere degradazioni per le prestazioni.

Questo progetto propone e implementa un sistema di **predictive auto-scaling**: un approccio in cui le decisioni di scaling vengono prese *prima* che il picco avvenga, sulla base di previsioni generate da un modello di serie temporali.

L'architettura è interamente basata su tecnologie open source e viene eseguita su un cluster Kubernetes locale (Minikube). I componenti principali sono:

- **Kubernetes** (Minikube) — orchestrazione dei container;
- **Prometheus** — raccolta e storage delle metriche;
- **KEDA** (Kubernetes Event-Driven Autoscaler) — motore di scaling basato su metriche esterne;
- **Prophet** (Meta) — modello di previsione per serie temporali;

1.1 Obiettivi

L'obiettivo principale del progetto è dimostrare che un sistema di scaling predittivo, riesce ad anticipare le variazioni di carico e ad allocare risorse in anticipo rispetto ai picchi, riducendo il rischio di saturazione dei pod rispetto all'approccio reattivo classico.

Capitolo 2

Architettura del Sistema

2.1 Visione d'insieme

Il sistema è composto da cinque componenti principali (*Load Generator* non è un componente effettivo del sistema, infatti viene usato solamente per simulare del carico), ciascuno deployato come pod Kubernetes indipendente. La figura 2.1 illustra i flussi di dati tra i componenti.

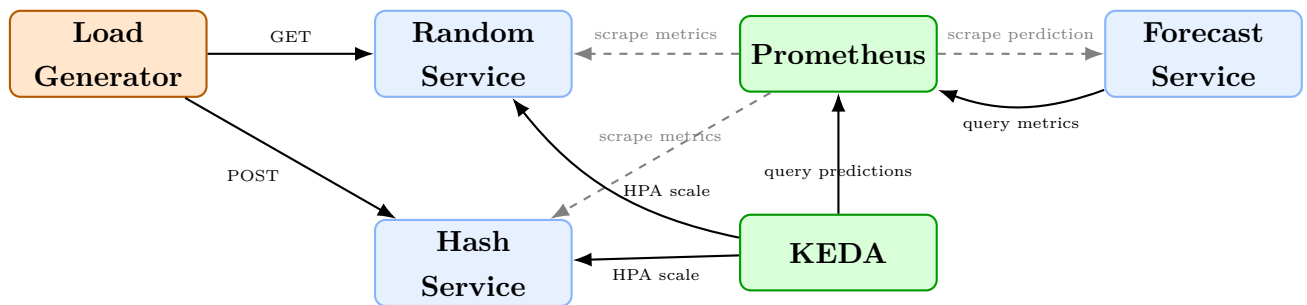


Figura 2.1: Architettura generale del sistema di predictive auto-scaling.

2.2 Struttura del progetto

Il repository è organizzato in cinque cartelle principali:

Directory	Contenuto
random-service/	Microservizio generatore di numeri casuali
hash-service/	Microservizio calcolatore di hash SHA-256
forecast-service/	Servizio di previsione con Prophet
load-generator/	Generatore di carico sinusoidale
k8s/	Manifest Kubernetes (Deployment, Service, ScaledObject, Ingress)

2.3 Componenti applicativi

2.3.1 Random Service

Il *Random Service* è un'applicazione Flask (porta 5000) che restituisce un numero intero casuale nell'intervallo $[0, 10^6]$ ad ogni richiesta GET. Espone il contatore delle richieste `random_requests_total` all'endpoint `/metrics`, consumato da Prometheus.

2.3.2 Hash Service

L'*Hash Service* è un'applicazione Flask (porta 5001) che calcola l'hash SHA-256 di una stringa inviata via POST. Anche questo servizio, come Random Service, espone il contatore delle richieste tramite `hash_requests_total`, all'endpoint `/metrics`.

2.3.3 Forecast Service

Il *Forecast Service* è il cuore del sistema predittivo. Opera su un ciclo di cinque minuti e svolge le seguenti operazioni:

1. Interroga Prometheus con le query PromQL `sum(rate(random_requests_total[5m]))` e `sum(rate(hash_requests_total[5m]))` per ottenere il traffico corrente;
2. Aggiunge i valori delle query in delle liste contenenti lo storico del traffico, ovviamente si hanno 2 liste differenti per i 2 servizi diversi;
3. Addestra un modello Prophet con stagionalità personalizzata a 30 minuti (la stagionalità è il modo in cui il modello cattura pattern periodici ricorrenti nel tempo all'interno di una serie temporale. Normalmente la stagionalità è giornaliera o settimanale, ma per la simulazione è scomodo aspettare così tanto);
4. Genera la previsione per il passo successivo (`FORECAST_STEPS = 1`, corrispondente a 5 minuti avanti);
5. Pubblica le previsioni come metriche Prometheus `predicted_random_rate` e `predicted_hash_rate`.

2.3.4 Load Generator

Il generatore di carico implementa un profilo sinusoidale configurabile tramite variabili d'ambiente. Il tasso di richieste al secondo segue la funzione:

$$\text{RPS}(t) = \frac{\text{MAX_RPS} + \text{MIN_RPS}}{2} + \frac{\text{MAX_RPS} - \text{MIN_RPS}}{2} \cdot \sin\left(\frac{2\pi t}{T}\right) \quad (2.1)$$

$T = \text{PERIOD_SECONDS} = 1800 \text{ s}$ (30 minuti), $\text{MIN_RPS} = 1$ e $\text{MAX_RPS} = 10$. Con i valori di default il traffico oscilla tra 1 e 10 richieste al secondo con un periodo di 30 minuti.

Il generatore distribuisce le richieste tra i due servizi in base al parametro `HASH_RATIO` (default 0.5): quindi metà delle richieste sono dirette al *Random Service* e l'altra metà all'*Hash Service*.

2.4 Infrastruttura Kubernetes

2.4.1 Networking e Ingress

I servizi applicativi sono esposti internamente come `ClusterIP` e resi accessibili dall'esterno tramite un Ingress NGINX e Ingress DNS con i seguenti hostname locali:

- `random.local` → Random Service (porta 80 → 5000)
- `hash.local` → Hash Service (porta 80 → 5001)
- `prometheus.local` → Prometheus Server (porta 80)

2.4.2 Prometheus

Legge le metriche esposte da Random Service, Hash Service e Forecast Service ogni 30 secondi (default) e permette la lettura delle metriche tramite query PromQL.

2.4.3 KEDA

Lo scaling è gestito da due risorse `ScaledObject` di KEDA, una per Random Service ed un'altra per Hash Service. Entrambe seguono lo stesso schema: KEDA interroga Prometheus ogni 30 secondi (default) e calcola il numero di repliche come:

$$\text{replicas} = \left\lceil \frac{\text{metricValue}}{\text{threshold}} \right\rceil \quad (2.2)$$

con `threshold = 1` req/s e metrica `avg_over_time(predicted*_rate[5m])`. Il numero di repliche è vincolato tra `minReplicaCount = 1` e `maxReplicaCount = 5`.

Capitolo 3

Implementazione

3.1 Microservizi Flask

Entrambi i microservizi applicativi condividono la stessa struttura: un'applicazione Flask che espone la logica sul path / e le metriche Prometheus sul path /metrics, tramite la libreria [prometheus_client](#).

```
1 @app.route("/")
2 def random_number():
3     REQUEST_COUNT.inc()
4     value = random.randint(0, 1000000)
5     return render_template_string(HTML_PAGE, number=value)
6
7 @app.route("/metrics")
8 def metrics():
9     return generate_latest(), 200, {"Content-Type":
    CONTENT_TYPE_LATEST}
```

Listing 3.1: Random Service (random-service/app.py)

```
1 @app.route("/", methods=["GET", "POST"])
2 def hash_service():
3     hash_value = None
4     if request.method == "POST":
5         REQUEST_COUNT.inc()
6         text = request.form.get("input_text", "")
7         data = text.encode()
8         data = hashlib.sha256(data).digest()
9         hash_value = data.hex()
10    return render_template_string(HTML_PAGE, hash_value=hash_value)
```

Listing 3.2: Hash Service (hash-service/app.py)

Invece il Forecast Service non ha bisogno di esporre la logica con Flask, infatti sono esposte solo le metriche:

```
1 @app.route("/metrics")
2 def metrics():
3     return generate_latest(), 200, {"Content-Type":
    CONTENT_TYPE_LATEST}
```

Listing 3.3: Forecast Service (forecast-service/forecast.py)

3.2 Forecast Service

3.2.1 Raccolta dati da Prometheus

Il servizio interroga Prometheus tramite l'API HTTP `/api/v1/query` con le query PromQL che calcolano il rate di richieste negli ultimi 5 minuti. L'uso di `rate()` rispetto al valore grezzo del counter garantisce che le metriche siano stabili al restart dei pod.

```
1 QUERY_RANDOM = "sum(rate(random_requests_total[5m]))"
2 QUERY_HASH   = "sum(rate(hash_requests_total[5m]))"
3
4 def query_prometheus(query):
5     r = requests.get(f"{PROM_URL}/api/v1/query", params={"query":
    query})
6     data = r.json()
7     if not data["data"]["result"]:
8         return 0.0
9     return float(data["data"]["result"][0]["value"][1])
```

Listing 3.4: Query a Prometheus (forecast-service/forecast.py)

3.2.2 Modello Prophet con stagionalità personalizzata

Prophet è un modello di previsione per serie temporali sviluppato da Meta. Decompone la serie in tre componenti:

$$y(t) = g(t) + s(t) + h(t) + \varepsilon_t \quad (3.1)$$

dove $g(t)$ è la tendenza (trend), $s(t)$ la stagionalità e $h(t)$ l'effetto delle festività (non utilizzato in questo contesto).

Il traffico simulato ha un periodo di 30 minuti, lontano dalle stagionalità default di Prophet (giornaliera e settimanale). Per questa ragione viene aggiunta una stagionalità personalizzata:

```
1 PERIOD_MINUTES = 30
2
3 def train_model(history):
4     if len(history) < 10:
5         return None
6     df = pd.DataFrame(history)
7     model = Prophet()
8     model.add_seasonality(
9         name='custom_cycle',
10        period=PERIOD_MINUTES / (60 * 24), # calcolato in giorni
11        fourier_order=5,
12    )
13    model.fit(df)
14    return model
```

Listing 3.5: Addestramento del modello (forecast-service/forecast.py)

3.2.3 Generazione della previsione

La funzione `forecast()` genera una previsione 5 minuti nel futuro (`FORECAST_STEPS = 1`) e restituisce il valore `yhat_upper`, il limite superiore dell'intervallo di confidenza. Per lo scaling preventivo è preferibile una stima leggermente ottimista, così da allocare risorse prima che la domanda raggiunga il picco.

```
1 FORECAST_STEPS = 1 # 1 passo = 5 minuti avanti
2
3 def forecast(model, history):
4     if model is None or len(history) < 10:
5         return history[-1]["y"]
6
7     future = model.make_future_dataframe(periods=FORECAST_STEPS,
8     freq="5min")
9     pred = model.predict(future)
10    return max(pred["yhat_upper"].iloc[-1], 0.0)
```

Listing 3.6: Generazione della previsione (forecast-service/forecast.py)

Se il modello non è ancora pronto o lo storico delle metriche è ancora troppo spoglio viene ritornato l'ultimo valore dello storico, in questo modo viene implementato uno *Scaling Reattivo* prima che si metta in funzione lo *Scaling predittivo*. Nella simulazione, per far vedere quando viene attivato veramente il modello, viene restituito `0.5`.

3.2.4 Loop principale

Il loop principale si ripete ogni 300 secondi (5 minuti). La durata del training di Prophet viene sottratta dal tempo di sleep per mantenere il ritmo costante.

```
1 def loop():
2     while True:
3         start_time = time.time()
4
5         random_metric = query_prometheus(QUERY_RANDOM)
6         hash_metric   = query_prometheus(QUERY_HASH)
7
8         add_point(history_random, random_metric)
9         add_point(history_hash,   hash_metric)
10
11        model_random = train_model(history_random)
12        model_hash   = train_model(history_hash)
13
14        pred_random = forecast(model_random, history_random)
15        pred_hash   = forecast(model_hash,   history_hash)
16
17        # Metriche di Prometheus
18        PRED_RANDOM.set(pred_random)
19        PRED_HASH.set(pred_hash)
20
21        elapsed = time.time() - start_time
22        time.sleep(max(0, 300 - elapsed))
```

Listing 3.7: Loop principale del Forecast Service

3.3 Configurazione KEDA

3.3.1 ScaledObject

Ogni servizio è associato a un `ScaledObject` KEDA che crea e gestisce automaticamente un HorizontalPodAutoscaler (HPA) Kubernetes. Il trigger di tipo `prometheus` interroga il server Prometheus e usa la media su 5 minuti della metrica predetta come valore di scaling.

Il meccanismo di scaling prevede:

- **Scale-up:** quando `avg_over_time(predicted_random_rate[5m]) > 1` req/s, KEDA aumenta le repliche proporzionalmente;
- **Scale-down:** quando la metrica scende sotto la soglia, KEDA riduce le repliche fino al minimo di 1;

- **Coldstart protection:** `minReplicaCount = 1` garantisce che almeno un'istanza sia sempre attiva.
- **Resource protection:** `maxReplicaCount = 5` massimo di repliche eseguibili nello stesso momento in modo da non consumare troppe risorse in caso di attacchi.

```
1 apiVersion: keda.sh/v1alpha1
2 kind: ScaledObject
3 metadata:
4   name: random-scaler
5 spec:
6   scaleTargetRef:
7     name: random-service
8   minReplicaCount: 1
9   maxReplicaCount: 5
10  triggers:
11  - type: prometheus
12    metadata:
13      serverAddress: http://prometheus-server.default.svc:80
14      metricName: predicted_random_rate
15      query: avg_over_time(predicted_random_rate[5m])
16      threshold: "1"
```

Listing 3.8: ScaledObject per il Random Service (k8s/random-scaler.yaml)

3.4 Generatore di Carico

```
1 def rps_sinusoidal(t: float) -> float:
2     amplitude = (MAX_RPS - MIN_RPS) / 2
3     midpoint = (MAX_RPS + MIN_RPS) / 2
4     return midpoint + amplitude * math.sin(2 * math.pi * t /
5     PERIOD_SECONDS)
6 def main():
7     while True:
8         elapsed = time.time() - _start_time
9         rps = max(0.1, rps_sinusoidal(elapsed))
10        n_requests = max(1, round(rps))
11        for _ in range(n_requests):
12            if random.random() < HASH_RATIO:
13                send_request(TARGET_HASH, "hash")
14            else:
15                send_request(TARGET_RANDOM, "random")
16        time.sleep(1.0)
```

Listing 3.9: Load Generator sinusoidale (load-generator/load_gen.py)

Capitolo 4

Risultati Sperimentali

4.1 Profilo di traffico generato

Il generatore di carico produce un profilo sinusoidale con le seguenti caratteristiche:

Parametro	Valore	Note
MIN_RPS	1 req/s	Valore minimo del ciclo
MAX_RPS	10 req/s	Valore massimo del ciclo
PERIOD_SECONDS	1800 s	Periodo di 30 minuti
HASH_RATIO	0.5	50% hash, 50% random

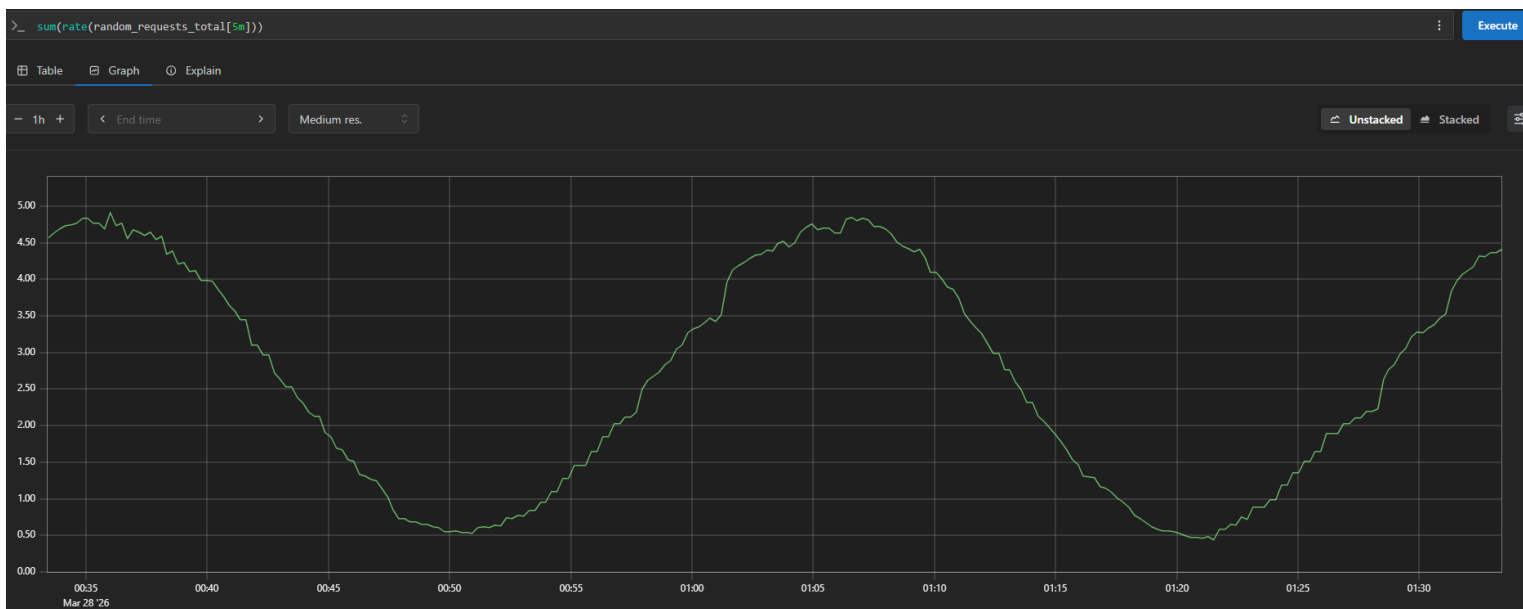


Figura 4.1: Traffico reale al Random Service misurato da Prometheus.

4.2 Metriche predette dal Forecast Service

Dopo un periodo di warm-up di circa 50 minuti (necessario ad accumulare almeno 10 campioni per il training di Prophet), il Forecast Service inizia a produrre previsioni. La metrica `predicted_random_rate` oscilla in anticipo rispetto al traffico reale, dimostrando la capacità predittiva del modello.

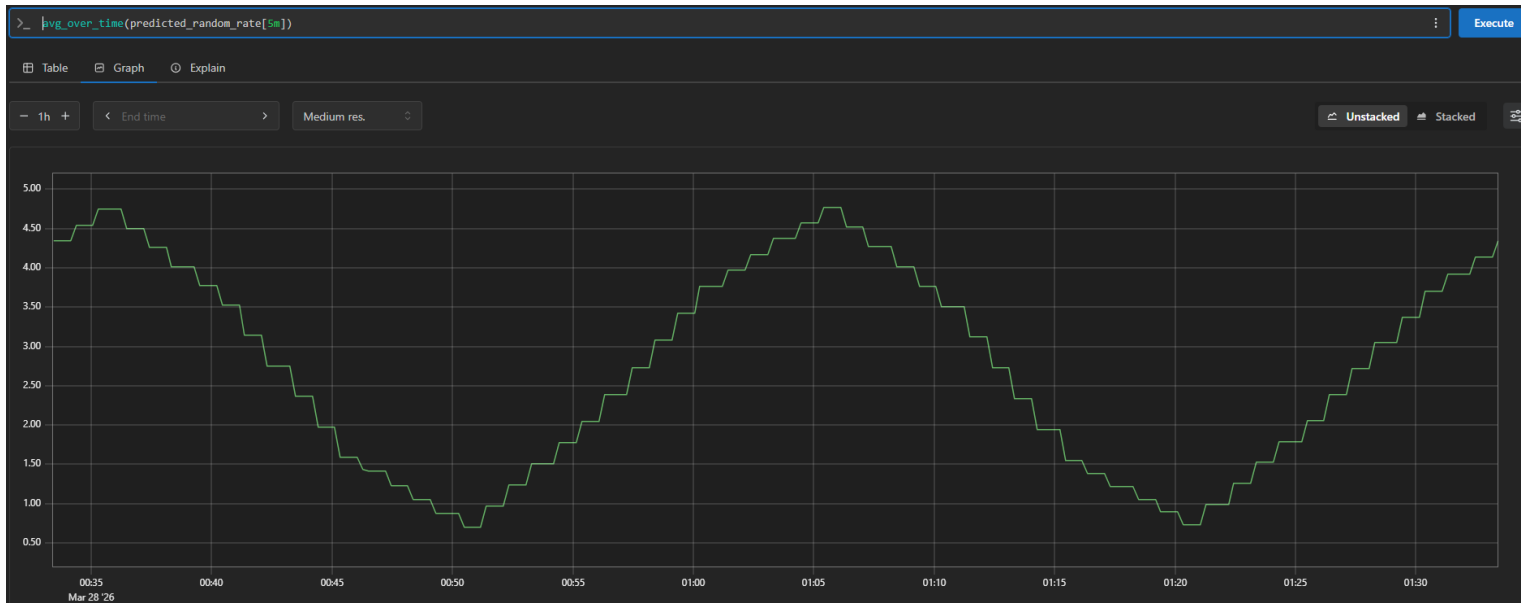


Figura 4.2: Metrica predetta `predicted_random_rate` esposta dal Forecast Service.

4.3 Comportamento dello scaling

4.3.1 Numero di repliche nel tempo

KEDA ha scalato il *Random Service* da 1 a 5 repliche durante i picchi di traffico.

4.3.2 Screenshot kubectl

```
1 $ kubectl get pod | grep "random"
2 random-service-7f8d87885d-2txlv      1/1      Running   0      3h45m
3 random-service-7f8d87885d-fcstp      1/1      Running   0      3h13m
4 random-service-7f8d87885d-kl2n8      1/1      Running   0      2m2s
5 random-service-7f8d87885d-mv2xd      1/1      Running   0      7m3s
6 random-service-7f8d87885d-z64px      1/1      Running   0      10m
```

Figura 4.3: Output di `kubectl get pod | grep "random"`: i pod del Random Service scalano da 1 a 5 repliche prima del picco di traffico.

```

1 > kubectl describe scaledobject random-scaler
2 Name:                random-scaler
3 Namespace:           default
4 Labels:               scaledobject.keda.sh/name=random-scaler
5 Annotations:          <none>
6 API Version:          keda.sh/v1alpha1
7 Kind:                 ScaledObject
8 Metadata:
9   Creation Timestamp:  2026-03-25T17:01:46Z
10  Finalizers:
11    finalizer.keda.sh
12  Generation:          2
13  Resource Version:    326207
14  UID:                  4403a2a5-528b-4e20-bb70-3ec0b5371428
15 Spec:
16   Min Replica Count:  1
17   Max Replica Count:  5
18   Scale Target Ref:
19     Name: random-service
20   Triggers:
21     Metadata:
22       Metric Name:    predicted_random_rate
23       Query:          avg_over_time(predicted_random_rate[5m])
24       Server Address: http://prometheus-server.default.svc:80
25       Threshold:      1
26     Type:             prometheus
27 Status:
28   Authentications Types:
29   Conditions:
30     Message: ScaledObject is defined correctly and is ready for scaling
31     Reason:   ScaledObjectReady
32     Status:   True
33     Type:     Ready
34     Message:  Scaling is performed because triggers are active
35     Reason:   ScalerActive
36     Status:   True
37     Type:     Active
38     Message:  No fallbacks are active on this scaled object
39     Reason:   NoFallbackFound
40     Status:   False
41     Type:     Fallback
42     Status:   False
43     Type:     Paused

```

```

1 External Metric Names:
2   s0-prometheus
3 Hpa Name:                keda-hpa-random-scaler
4 Last Active Time:        2026-03-28T00:36:35Z
5 Original Replica Count:  1
6 Scale Target GVKR:
7   Group:                  apps
8   Kind:                   Deployment
9   Resource:               deployments
10  Version:                 v1
11 Scale Target Kind:      apps/v1.Deployment
12 Triggers Activity:
13   s0-prometheus:
14     Is Active:            true
15 Triggers Types:         prometheus
16 Events:                 <none>

```

Figura 4.4: Stato del ScaledObject KEDA con trigger attivo.

4.4 Interfacce utente dei servizi

(a) Random Service (`random.local`)(b) Hash Service (`hash.local`)

Figura 4.5: Interfacce web dei microservizi applicativi.

4.5 Analisi del modello Prophet

4.5.1 Stagionalità rilevata

La stagionalità personalizzata a 30 minuti, definita con `fourier_order = 5`, permette a Prophet di approssimare la forma sinusoidale del traffico con sufficiente flessibilità senza overfitting. Un valore più alto di `fourier_order` aumenta la flessibilità del modello ma può portare a instabilità nelle previsioni. Dato che come esempio viene usato un carico sinusoidale, quindi molto stabile, è meglio abbassare l'ordine di fourier per evitare previsioni falsate (il valore di default è 10).

4.5.2 Scelta di `yhat_upper`

L'utilizzo del limite superiore dell'intervallo di confidenza (`yhat_upper`) anziché della stima centrale (`yhat`) introduce una sovrastima controllata che favorisce lo scale-up preventivo. Questo comportamento è desiderabile in un sistema di predictive scaling: è preferibile avere risorse leggermente in eccesso prima del picco piuttosto che risorse insufficienti durante il picco stesso.

Capitolo 5

Conclusioni

Il progetto ha dimostrato la fattibilità di un sistema di predictive auto-scaling interamente basato su componenti open source, deployato su un cluster Kubernetes locale. I risultati ottenuti mostrano che:

1. Il **Forecast Service** è in grado di apprendere il pattern sinusoidale del traffico e di generare previsioni coerenti con l'andamento reale, dopo un periodo iniziale di warm-up;
2. **KEDA** consuma correttamente le metriche predette esposte su Prometheus e scala i deployment in anticipo rispetto ai picchi di traffico;
3. L'uso di `yhat_upper` come metrica di scaling introduce un margine di sicurezza che riduce il rischio di saturazione durante le fasi di crescita rapida del carico;
4. Il **profilo sinusoidale** del generatore di carico si è rivelato particolarmente adatto per Prophet, che eccelle nel riconoscere stagionalità periodiche regolari.

5.1 Possibili evoluzioni

Il sistema nella sua forma attuale rappresenta una prova di concetto. Tra le possibili estensioni si possono identificare:

- **Multi-step forecasting:** aumentare `FORECAST_STEPS` per predire con maggiore anticipo, al costo di una potenziale riduzione di accuratezza;
- **Retraining incrementale:** ottimizzare il training di Prophet usando tecniche di warm-starting per ridurre il tempo di computazione;
- **Modelli alternativi:** sperimentare con LSTM o altri modelli di deep learning per la previsione di serie temporali più irregolari.

Bibliografia

- [1] KEDA Project, *Kubernetes Event-Driven Autoscaling — Official Documentation*,
<https://keda.sh/docs/>
- [2] Meta Open Source, *Prophet: Forecasting at Scale*,
https://facebook.github.io/prophet/docs/quick_start.html
- [3] Prometheus, *Monitoring System and Time Series Database*,
<https://prometheus.io/docs/introduction/overview/>
- [4] Kubernetes, *Run Kubernetes Locally*,
<https://minikube.sigs.k8s.io/docs/start/>